

Chapter 1

Data Quality, Cleaning, and Trustworthy Variables

1.1 Why Data Quality Comes Before Modeling

Before we build a statistical model, compute a regression line, or compare prediction errors, we need to know whether the variables in the data set can be trusted. Exploratory data analysis begins with the habit of “listening to the data.” That means stepping back from the final question we want to answer and first asking what the data are trying to tell us about themselves.

Data cleaning is not a clerical step that happens before the real analysis. It is part of the analysis. A model can only learn from the variables we give it. If those variables are mislabeled, inconsistently recorded, stored in the wrong type, or contaminated by impossible values, then the model may produce output that looks mathematical but is not analytically defensible.

This chapter focuses on the practical question:

What must be true about our variables before we trust them enough to summarize, visualize, or model them?

A trustworthy variable is one whose meaning, type, units, allowable values, and missing-value behavior are understood well enough that another analyst could reproduce the cleaning decision and defend it to a customer, commander, or reviewer. The goal is not to make the data look perfect. The goal is to make the workflow repeatable and the decisions defensible.

1.2 Data Tables, Observations, and Variables

A data set is often organized as a rectangular table. The rows are observations, and the columns are variables. If there are n observations and p variables, we can represent the data as a matrix

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix} \in \mathbb{R}^{n \times p} \quad (1.1)$$

when all entries are numeric. The entry x_{ij} is the value of variable j for observation i .

The i th row,

$$\mathbf{x}_i^T = (x_{i1}, x_{i2}, \dots, x_{ip}),$$

contains all measured information for observation i . The j th column,

$$\mathbf{x}_j = (x_{1j}, x_{2j}, \dots, x_{nj})^T,$$

contains all observed values of variable j .

This matrix notation is useful, but it can also hide an important fact: not every variable is genuinely numeric. Some columns may contain categories, ordered labels, dates, text, identifiers, or codes. A column made of numbers is not automatically a numerical variable in the statistical sense.

Math Foundations Connection. In linear algebra, a vector is a collection of entries that pertain to the same object or concept. In data analysis, a numeric variable can be viewed as a vector of observed values, and a full numeric data table can be viewed as a matrix whose columns are variable vectors. This perspective helps later when we discuss design matrices, regression, and projections. For cleaning, it helps us remember that operations applied to a column should make sense for the type of object that column represents. See the Math Foundations textbook, Chapters 2, 4, and 7.

1.3 Variable Types

A variable is an attribute that may change its value between observations in a data set. The type of a variable determines which summaries, plots, transformations, and models are meaningful.

Variable type	Meaning	Natural summaries
Continuous numeric	Values can be thought of as varying across a range, rather than only at isolated counts. The observed minimum and maximum are sample bounds, not part of the definition.	mean, median, spread, histogram
Discrete numeric	Values are usually counts or integer-valued measurements.	counts, mean, spread, bar plot or histogram
Ordinal	Categories have an order, but distances between categories may not be meaningful.	counts, proportions, median, ordered bar plot
Categorical	Categories are labels without a natural ordering.	counts, proportions, mode, bar plot

Examples of continuous numeric variables include age, temperature, height, time to complete a task, and distance traveled. Examples of discrete numeric variables include number of missions completed, number of defects found during inspection, and number of officers in a unit. Enlisted rank is an ordinal variable: the order matters, but the numerical distance from one rank to the next is not usually treated as constant. Service branch, blood type, platform type, and property view are categorical variables.

A common trap is treating numeric codes as quantitative measurements. A survey might record service branch as Army = 1, Marine Corps = 2, Navy = 3, Air Force = 4, Space Force = 5, and Coast Guard = 6. Those numbers are convenient labels, not measurements. The average of Army

and Navy is not Marine Corps, and the value 6 is not larger than the value 1 in a quantitative sense. Later, categorical variables can be encoded numerically for modeling, but those encodings are modeling tools rather than evidence that the original category has arithmetic meaning.

1.4 Why Data Become Messy

Data can be messy for many reasons. Some problems are obvious, such as impossible values. Others are subtle, such as inconsistent units or category codes that look numeric. Common sources of data problems include data-entry errors, non-standardized labels, missing or incomplete data, inconsistent survey responses, inconsistent units or scales, duplicate records, impossible values, incorrect data types, dates stored as text, categorical variables stored as integers, and numeric variables stored as strings.

The analyst’s job is not merely to find these issues. The analyst must decide what to do about them and document the decision well enough that someone else could follow the same logic.

1.5 The Data Cleaning Workflow

Data cleaning is the process of identifying, documenting, and correcting data-quality issues before analysis. A useful first-pass workflow is:

1. identify and handle missing values,
2. fix data types,
3. rename and standardize columns,
4. subset rows and columns as needed,
5. detect outliers,
6. conduct basic sanity checks.

This sequence is not strictly linear. Fixing a data type may reveal new missing values; detecting an outlier may reveal a unit mismatch; a sanity check may send you back to the original data source. The workflow is iterative.

A cleaning decision should satisfy two standards. First, it should be *repeatable*: the same code plus the same raw data should produce the same cleaned data. Second, it should be *defensible*: the analyst should be able to explain why the decision was reasonable for the variable, the data source, and the purpose of the analysis.

A minimal cleaning log should record the variable, the issue, the decision, and enough detail for another analyst to reproduce the choice. For example, “**weight_lb**: values less than or equal to 0 are physically impossible for the study population; recoded as missing.” The log does not need to be elaborate, but it should prevent the cleaning process from becoming a set of undocumented manual edits.

1.6 Missing Values and Imputation

Missing values occur when an observation should have a value for a variable but the recorded data do not contain that value. Missingness can be explicit, such as `NA`, `NaN`, blank cells, or `NULL`. Missingness can also be implicit, such as a value of 0 used as a placeholder for an impossible measurement.

For a data matrix, define a missingness indicator

$$m_{ij} = \begin{cases} 1, & \text{if } x_{ij} \text{ is missing,} \\ 0, & \text{if } x_{ij} \text{ is observed.} \end{cases} \quad (1.2)$$

Then the number of missing values in variable j is

$$M_j = \sum_{i=1}^n m_{ij}, \quad (1.3)$$

and the missingness rate is

$$r_j = \frac{M_j}{n}. \quad (1.4)$$

These simple counts identify which variables have missingness and how severe it is.

One of the most important missing-data checks is distinguishing *true zeros* from *missing zeros*. A true zero means the value is genuinely zero, such as `sibling_count = 0`. A missing zero means zero was used as a placeholder for a missing or invalid value, such as `weight_lb = 0` for an adult human. Missingness cannot be detected by a universal rule such as “replace all zeros.” The rule must depend on the variable definition.

Once missing values are identified, the analyst usually has three choices: drop, impute, or flag.

Action	When it may be reasonable	Main risk
Drop rows	Missingness is limited, the variables are required, and the complete rows are still representative.	Can remove useful data or bias the analysis.
Drop columns	A variable is unnecessary, too incomplete, unreliable, or poorly defined.	May discard a useful signal for a later analysis.
Impute	A missing value can be replaced using a defensible rule or model.	Can create false precision or distort distributions and relationships.
Flag missingness	Missingness itself may be informative.	Requires careful interpretation in summaries and models.

For complete-case analysis, if J is the set of variables required for a particular analysis, the retained row set is

$$I_{\text{complete}} = \{i : m_{ij} = 0 \text{ for all } j \in J\}. \quad (1.5)$$

The cleaned analysis table is the submatrix containing rows in I_{complete} and columns in J .

Imputation replaces missing values with reasonable substitutes. If x_{ij} is missing, an imputed value is often written as $\hat{x}_{ij}$, giving

$$x_{ij}^{\text{clean}} = \begin{cases} x_{ij}, & \text{if } x_{ij} \text{ is observed,} \\ \hat{x}_{ij}, & \text{if } x_{ij} \text{ is missing.} \end{cases} \quad (1.6)$$

Simple imputation uses only the variable with missingness. For a numeric variable X_j , mean imputation replaces missing values with

$$\hat{x}_{ij} = \bar{x}_{j,\text{obs}} = \frac{1}{n - M_j} \sum_{i:m_{ij}=0} x_{ij}. \quad (1.7)$$

Median, mode, constant, or explicit-category imputation may also be used when appropriate. Conditional imputation uses other variables to make better guesses, such as regression imputation, k -nearest-neighbor imputation, predictive mean matching, or other model-based methods. Multiple imputation creates several plausible completed data sets, analyzes each, and combines the results.

The key caution is that imputation is an assumption, not a fact. Avoid imputing automatically when “did not answer” is itself meaningful, when the variable is not needed, when missingness is too extensive, when imputation would introduce obvious bias, when the missing value represents a choice rather than a failed measurement, or when the data source is too unreliable to support replacement.

1.7 Fixing Data Types and Standardizing Variables

A software data type is how software stores and treats a variable. A statistical variable type is what the variable means analytically. Cleaning often requires aligning both.

Common data-type problems include numeric columns stored as strings, categorical variables stored as integers, dates stored as text, binary variables stored as text labels, identifiers stored as numbers, and numbers with commas, units, or symbols stored as text. The practical cleaning question is:

Does the software representation support the analysis we intend to perform?

Problem	Examples	Cleaning check
Numeric stored as text	1,250, 72 in, unknown, --	Parse valid numbers; decide whether nonnumeric entries are missingness, labels, or errors.
Category stored as integer	Service branch coded 1–6	Ask whether arithmetic on the codes is meaningful. If not, treat as categorical or ordinal.
Date stored as text	04/03/2026, 2026-04-03, 3 Apr 26	Parse to date format, then validate impossible dates, unexpected future dates, and time-zone issues.
Identifier stored as number	ID, hull number, survey response number	Preserve identity; do not average or model as a quantitative variable unless explicitly meaningful.

Column names are also part of the analysis. A useful naming convention should be consistent, descriptive, short enough to use in code, free of spaces and special characters, and stable across related data sets. Including units in the name can prevent later confusion: `height_in` and `height_cm` are safer than simply `height`.

Subsetting means selecting only part of a data set. Let $I \subseteq \{1, 2, \dots, n\}$ be a set of row indices and let $J \subseteq \{1, 2, \dots, p\}$ be a set of column indices. The submatrix

$$\mathbf{X}_{I,J}$$

keeps rows in I and columns in J . For example, if $x_{i,\text{age}}$ is the age for observation i , then

$$I_{\text{adult}} = \{i : x_{i,\text{age}} \geq 18\} \quad (1.8)$$

keeps adult observations. Column subsets are usually defined by the analysis purpose, such as keeping the response, the intended predictors, and identifiers needed for traceability.

Math Foundations Connection. Subsetting is a set operation. The row set I identifies which observations remain. The column set J identifies which variables remain. Thinking in sets helps avoid vague language such as “we removed bad rows.” A defensible cleaning rule should be expressible as a clear predicate, such as “remove rows where weight is missing or impossible.” See the Math Foundations textbook, Chapters 2 and 4.

1.8 Outliers and Sanity Checks

An outlier is an observation that is unusually far from the rest of the data or does not follow the overall pattern. Outliers matter because they can strongly affect summaries, visualizations, and fitted models. They should be examined before computing final summaries or fitting final models.

Common visual tools include histograms, boxplots, and scatterplots. A histogram can reveal extreme values, unusual gaps, skewness, or multiple modes. A boxplot can reveal observations outside the central bulk of the data. A scatterplot can reveal points that do not follow a bivariate pattern.

Outliers are not automatically errors. An outlier may be a data-entry error, a unit-conversion error, a duplicate or merged-record problem, a rare but valid observation, evidence of subgroups, or evidence that the model or summary is too simple. Removing a valid extreme observation just because it is inconvenient can bias the analysis.

A common numerical rule for outlier flagging uses the interquartile range. Let Q_1 be the first quartile, Q_3 be the third quartile, and

$$\text{IQR} = Q_3 - Q_1. \quad (1.9)$$

Tukey fences flag values outside

$$[Q_1 - 1.5 \text{ IQR}, Q_3 + 1.5 \text{ IQR}]. \quad (1.10)$$

This rule is a flag, not an automatic deletion command.

Sanity checks ask whether the data make sense before we treat them as evidence. One useful warning is:

Be wary of taking data at face value, especially when you do not know how it was generated.

At minimum, ask:

- Are units consistent, or might a variable mix inches with centimeters, miles with kilometers, or dollars with thousands of dollars?
- Are there impossible values, such as negative ages, probabilities outside $[0, 1]$, counts below zero, or physically impossible measurements?

- Does the number of rows match the expected row unit: one person, one transaction, one unit, one day, or one person-day?
- Are duplicates expected or suspicious?
- Are categorical levels spelled consistently?
- Do minimums, maximums, histograms, and summary statistics agree with subject-matter expectations?

Impossible values are not always easy to repair. Sometimes the correct value can be recovered from documentation or another field. Sometimes the value should be treated as missing. Sometimes the entire record should be excluded from a specific analysis. The decision should be documented.

1.9 A Trustworthy Variable Checklist

Before using a variable in a summary, plot, or model, ask the following questions:

1. **Definition:** What does the variable measure, and what is the row unit associated with the measurement?
2. **Type:** Is the variable continuous, discrete, ordinal, categorical, date/time, text, or an identifier? Does the software type match the statistical meaning?
3. **Units and scale:** Are units consistent? Are values raw, standardized, logged, percentages, or rates?
4. **Missingness:** What values indicate missingness? Are placeholders such as 0, 999, -1, or blanks being used? Is missingness itself meaningful?
5. **Validity:** What values are possible, impossible, or unusual but possible? Are there outliers, duplicates, or inconsistent labels?
6. **Defensibility:** Can the cleaning decision be explained in plain language, implemented in code, and documented in a cleaning log?

This checklist is intentionally practical. A variable is not trustworthy merely because it has numbers in a column. It becomes trustworthy when its meaning and cleaning decisions are clear enough to reproduce and defend.

1.10 Realities of Data Cleaning

Real data-driven studies often involve very large data sets. In an ideal world, errors would be fixed at the data-entry point. In practice, analysts often receive data after many upstream choices have already been made. We fix what we can, document what we cannot fix, and avoid pretending that the data are cleaner than they are.

Some issues are easier to spot than others. Numerical problems may be visible through summary statistics and visual EDA. Categorical problems may require careful inspection of labels and codes. Missingness may require knowledge of how the data were collected. Credibility issues may require comparing the data against external expectations or subject-matter knowledge.

Errors can also be informative. If a toy example contains several obvious errors, those errors may be a warning about the credibility of the data source. The right conclusion may not be “clean harder.” It may be “find a better data set.”

1.11 Representative Mini-Examples

Two common cleaning traps illustrate the chapter’s main ideas.

Categorical codes are not quantities. Suppose a survey records service branch using integer codes. A careless analysis might treat the column as numeric and compute an average branch code. That summary is meaningless. The defensible cleaning decision is to recode the integers to meaningful labels and treat the variable as categorical.

Zero may mean different things in different variables. A value of `sibling_count` = 0 may be a valid true zero. A value of `weight_lb` = 0 for an adult is likely missing or invalid. The same recorded number has different meanings because the variables have different definitions. Cleaning rules must be variable-specific.

1.12 Math Foundations Source Map

This source map records the Math Foundations support used in this chapter and where those ideas appear in the completed Math Foundations textbook.

Chapter 3 use	Math Foundations connection	MF textbook location
Data rows and variables as structured vectors	$\mathbb{R}^n$ vector notation, entries, subvectors, and vectors as collections of quantities	Ch02 Secs. 2.1, 2.2, 2.6
Numeric, ordinal, and categorical variable domains	Set membership, complements, set difference, subsets, and common number systems	Ch02 Sec. 2.6; Ch04 Secs. 4.1, 4.2, 4.4
Filtering rows and selecting columns	Subvector/slice notation and matrix submatrix extraction	Ch02 Sec. 2.2; Ch07 Secs. 7.1, 7.2
Cleaning by subsetting observations	Set operations, subset notation, quantified statements, and negation of quantified statements	Ch01 Sec. 1.1; Ch02 Sec. 2.6; Ch04 Secs. 4.1, 4.2, 4.4, 4.7, 4.8
Outlier detection as distance from typical pattern	Euclidean norm, distance between vectors, and angle/orthogonality intuition	Ch05 Secs. 5.5, 5.7, 5.8
Design-matrix readiness after cleaning	Matrix dimensions, sparse matrices, transpose notation, and matrix-vector multiplication	Ch07 Secs. 7.1, 7.4, 7.6.1, 7.6.2

1.13 Chapter Summary

Data cleaning is the foundation for trustworthy data analysis. Before modeling, the analyst must understand what each variable means, what type it is, how it is stored, what values are possible, and how missingness is represented.

The main ideas from this chapter are:

- Exploratory data analysis begins by listening to the data before asking targeted modeling questions.
- Variables may be continuous, discrete, ordinal, or categorical.
- Numeric storage does not guarantee numeric meaning.
- Missing values must be detected, counted, interpreted, and handled deliberately.
- True zeros must be distinguished from placeholder zeros.
- Dropping data is simple but can remove information or introduce bias.
- Imputation can preserve rows but introduces assumptions.
- Data types, column names, row subsets, and outliers must be checked systematically.
- Sanity checks protect the analyst from taking unreliable data at face value.
- Cleaning decisions should be repeatable and defensible.

The next chapter builds on this foundation by using numerical summaries and visual structure to describe cleaned data. Once variables are trustworthy enough to inspect, we can ask what their distributions, relationships, and patterns reveal.